

to salient feature values. Each case thus defined is a proto-instance that can be instantiated time and time again by different trainees in different simulation runs. As a concrete example, our Maryland Virtual Patient clinician training application featured an inventory of virtual patients, each of which was a proto-instance. In training systems, proto-instances allow teachers to encapsulate different teaching scenarios, such as a GERD patient who will progress to adenocarcinoma if left untreated.

### 3.2.4 Scripts

Ontological scripts record complex events along with their participants and props.<sup>19</sup> They can reflect knowledge in any domain—what happens at a doctor’s appointment, how to build a chair, what to do at a four-way stop; and they can be at any level of specificity—from a basic sequence of events to the level of detail needed to generate a computer simulation. Their descriptions include more expressive means than the simple frame illustrated by SURGERY above. For example, scripts can require the coreferencing of arguments across events, they can have optional and variously ordered events, they can require time management, and so on. Scripts can be recorded by knowledge engineers, or they can be acquired by agents on the fly.<sup>20</sup>

Scripts both guide agent operation and support their reasoning about the world. Example (3.5) illustrates how script-based knowledge is needed for making implicatures during language understanding.

(3.5) “How was your doctor’s appointment?” “Great! The scale was broken!”

Why does the second speaker say *the scale*? What licenses the use of *the*, considering that this object was not previously introduced into the discourse? The mention of a doctor’s appointment prepares the listener to mentally access objects, like scale, and events, like getting weighed, that are typically associated with a doctor’s appointment, making those objects and events primed for inclusion in the situation model. In fact, the linguistic licensing of *the* with *scale* is evidence that such script activation actually takes place. Of course, it is script-based knowledge that also explains why the person is happy—and it further allows us to infer the body type of the speaker. If we want LEIAs to be able to reason at this level as well, then scripts are the place to store the associated knowledge.

When the knowledge in a script is used to guide agent action, the agent must create an instance of it, which is called a plan. A plan differs from a script in that (a) it selects a particular path through the often-variable sequence of events permitted by the script and (b) it fills the events' case roles in particular ways.

The events and objects referred to in a script are, themselves, concepts that are recorded in appropriate branches of the ontology. This means that scripts are organized like well-constructed computer programs—not as a massive main function but, rather, as hierarchically organized drilldowns of scripts that ultimately end in singleton events or function calls.

We will describe scripts using the example of the AGENT-FUNCTIONING-FLOW script that implements the agent's cognitive architecture and makes the agent self-aware so that it can explain its own functioning (cf. section 2.1). This script uses the EVENTS and OBJECTS shown in the ontological subtrees in table 3.2, which we provide as a crib to be consulted when reading the scripts themselves. This description of AGENT-FUNCTIONING-FLOW serves the double duty of presenting an example of a script as an ontological entity and describing an important aspect of LEIA operation.

The top level of the AGENT-FUNCTIONING-FLOW script has five ordered subevents that correlate with the architecture diagram in figure 2.1.<sup>21</sup>

AGENT-FUNCTIONING-FLOW

DEFINITION This script implements the LEIA's cognitive architecture.

AGENT LEIA-1

SUBEVENTS

- 1. PERCEPTION-RECOGNITION
- 2. PERCEPTION-INTERPRETATION
- 3. DELIBERATION
- 4. ACTION-SPECIFICATION
- 5. ACTION-RENDERING

All of the subevents are, themselves, scripts, which we will describe in turn. The first one is PERCEPTION-RECOGNITION, whose function is described in its definition field.

PERCEPTION-RECOGNITION

DEFINITION The agent determines which recognizer is needed to process a signal, runs it, and stores the resulting data.

AGENT LEIA

INPUT PERCEPTION-INPUT

OUTPUT PERCEIVED-DATA

SUBEVENTS

- TRY: recognize-text-input
- EXPL “text input is recognized and stored as data”
- TRY: recognize-speech
- EXPL “speech input is recognized and stored as data”
- TRY: recognize-visual-input
- EXPL “visual input is recognized and stored as data”
- TRY: recognize-interoception
- EXPL “interoceptive input is recognized and stored as data”

The input to PERCEPTION-RECOGNITION is any kind of PERCEPTION-INPUT which, as indicated by that concept’s subclasses in table 3.2, currently includes

**Table 3.2**  
The ontological subtrees of EVENTS and OBJECTS that are used in the AGENT-FUNCTIONING-FLOW script.

| EVENTS                         | OBJECTS                      |
|--------------------------------|------------------------------|
| AGENT-FUNCTIONING              | AGENT-SPECIFIC-OBJECTS       |
| AGENT-FUNCTIONING-FLOW         | AGENT-COGNITION-TOOL         |
| AGENT-PERCEPTION-EVENT         | INTEROCEPTION-PROCESSOR      |
| PERCEPTION-RECOGNITION         | NLU-SYSTEM                   |
| PERCEPTION-INTERPRETATION      | VISION-PROCESSOR             |
| INTEROCEPTION                  | PERCEPTION-INPUT             |
| NATURAL-LANGUAGE-UNDERSTANDING | INTEROCEPTION-SIGNAL         |
| VISION-INTERPRETATION          | SPEECH-SIGNAL                |
| AGENT-REASONING-EVENT          | TEXT-INPUT                   |
| AGENT-PLANNING                 | VISION-INPUT                 |
| MEMORY-MANAGEMENT              | PERCEIVED-DATA               |
| DELIBERATION                   | INTEROCEPTION-DATA           |
| PROCESS-DAEMONS                | SPEECH-DATA                  |
| AGENT-ACTION-EVENT             | TEXT-DATA                    |
| ACTION-SPECIFICATION           | VISION-DATA                  |
| CONVERT-MMR-TO-GMR             | XMR ; meaning representation |
| CONVERT-MMR-TO-AMR             | AMR ; robotic action MR      |
| CONVERT-MMR-TO-SMR             | GMR ; generation MR          |
| RENDERING                      | IMR ; interoception MR       |
| GENERATE-GMR                   | MMR ; mental MR              |
| LAUNCH-ROBOTIC-EFFECTOR        | SMR ; simulated action MR    |
| SIMULATE-PHYSICAL-ACTION       | TMR ; text MR                |
|                                | VMR ; vision MR              |

interoception, speech, text, and vision. The output is the associated kind of PERCEIVED-DATA, whose subclasses are also shown in the table.

There are four subevents, which are actually conditions with different preconditions that are evaluated in turn; this is the semantics of “TRY.” The SUBEVENTS field says, “If this is text input, then do text-input recognition; Else if this is speech input, then do speech-input recognition,” and so on. These subevents are not concepts, they are pointers to code that carries out the associated functions. Their status as pointers to code is indicated by the lowercase font. Since they are not concepts, the agent cannot look up their definitions to know what they mean and what they do. So, to enable the agent to explain these actions, a metadata field called “EXPL” (explanation) provides a short description.

**Using concepts vs. procedure calls as subevents of scripts**

A subevent of a script is recorded as a concept if:

- a. it is, itself, a script; or
- b. it is a non-decomposable event that has a freestanding status in the ontology.

By contrast, a subevent of a script is recorded as a procedure call if:

- a. it implements a procedure that is below the threshold of what the agent needs to understand;
- b. it implements a procedure that is not explainable because it is grounded in machine learning; or
- c. it implements a procedure that should eventually be described using a concept but is temporarily being treated as an opaque function in order to speed up the implementation of a particular system.

It is important, methodologically, not to attempt to make every line of code needed to implement LEIAs fully understood and explainable by them. This would be an inefficient use of resources. Instead, agents should be self-aware to a useful degree, and they should be prepared to explain their behavior in useful ways.

When the agent instantiates PERCEPTION-RECOGNITION as a plan, that plan reflects a specific path through the script, depending on which type of input was recognized. If a language input was recognized, the agent launches NATURAL-LANGUAGE-UNDERSTANDING; if a visual input was recognized, the agent launches VISION-INTERPRETATION; if a bodily sensation was recognized, the

Let us recap some important points about scripts.

- Scripts can be recorded by knowledge engineers, or they can be learned by agents during their operation.
- Scripts neatly open up into subscripts, which organizes knowledge representation.
- Scripts are as concept-based and explainable as possible but as streamlined and program-based as necessary. It would be a poor use of time to attempt to make every line of code that implements agent action fully explainable by the agent.
- When a script is instantiated as a plan, it reflects a specific path through the script and requires such things as handling coreferences of participants and props across multiple events.
- Like all of the agent's knowledge, scripts are fully inspectable and modifiable over time.
- When scripts are acquired by knowledge engineers, this can involve collaboration by system engineers (cf. section 2.5.2). This is particularly important for scripts that contain a significant amount of unexplainable code, such as those involving time management in simulation.
- Whereas more straightforward scripts can be learned by agents, those that reflect the mental models of domain experts are most efficiently modeled by knowledge engineers collaborating with those experts (see section 9.2). However, agents can automatically update such models, as by learning about new therapies for an already-modeled disease from the literature.<sup>22</sup>
- Agents can explain scripts based on:
  - their understanding of the basic shapes of scripts:

Numbered events occur in order, conditions (introduced by TRY) are ordered if-then statements, and so on. We did not present all script-related conventions here since that would be excessive detail for non-developers;
  - the natural language definitions of the concepts that comprise a script:

Agents can use definitions directly in explanations or they can semantically analyze them as part of a more reasoning-heavy explanation process. Definitions might be missing for scripts that agents learn without the involvement of language;